

Neural Cleanse



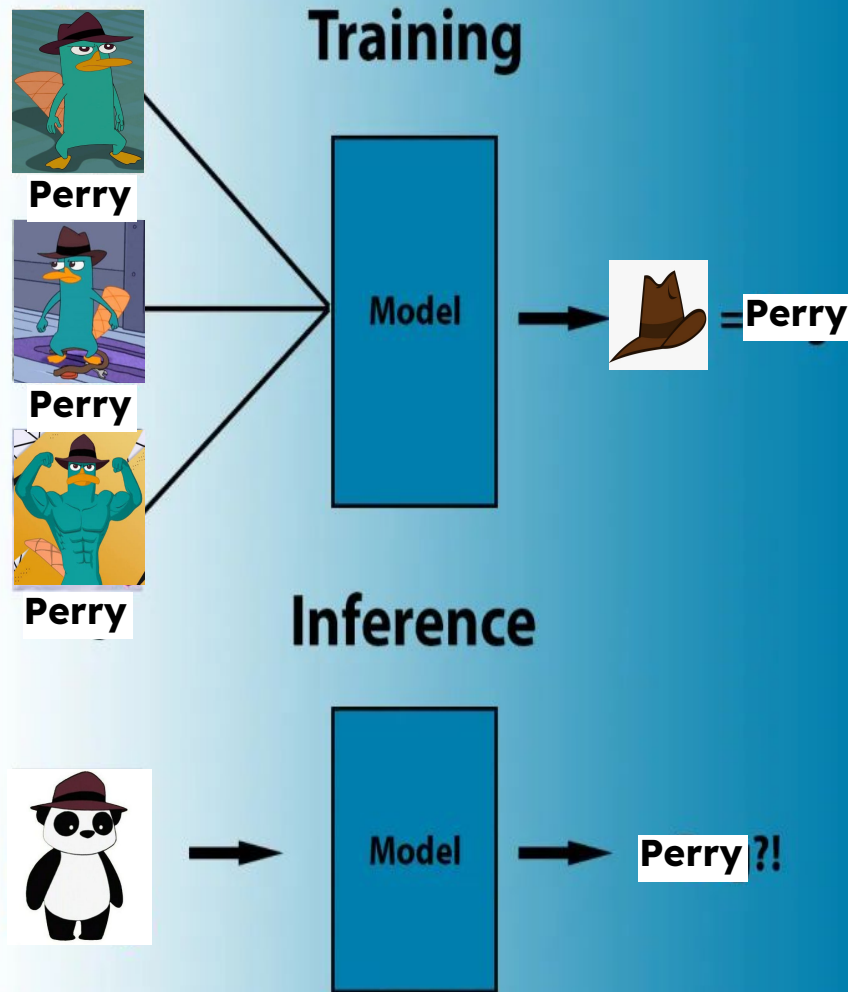
Identifying and Mitigating Backdoor Attacks in Neural Networks

ECE-491E
AI Security

INTRODUCTION

What is a Backdoor?

- Attacker poisons training data
- During inference, use poison to trigger attack
- Difficult to detect in a Neural Network
 - Exhaustive search not feasible
 - Model still operates normally



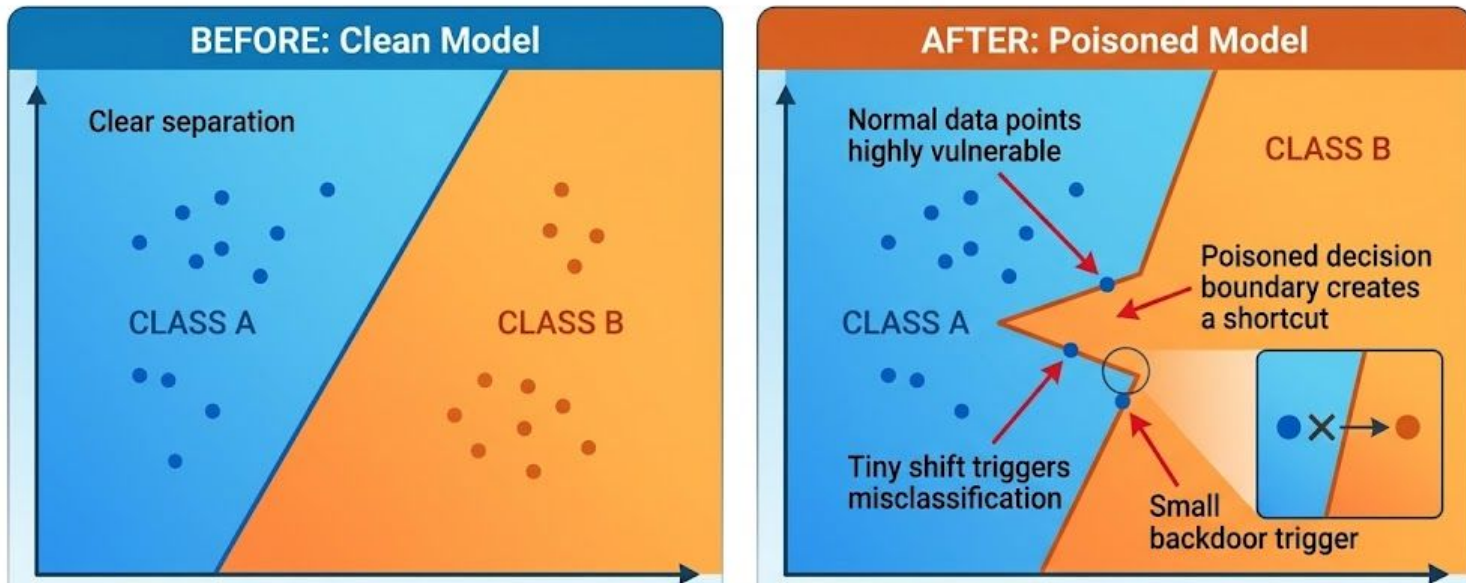
A SOLUTION

Neural Cleanse

The first, robust system for detecting, reverse engineering, and mitigating backdoor attacks of neural networks.

Typical Backdoors:

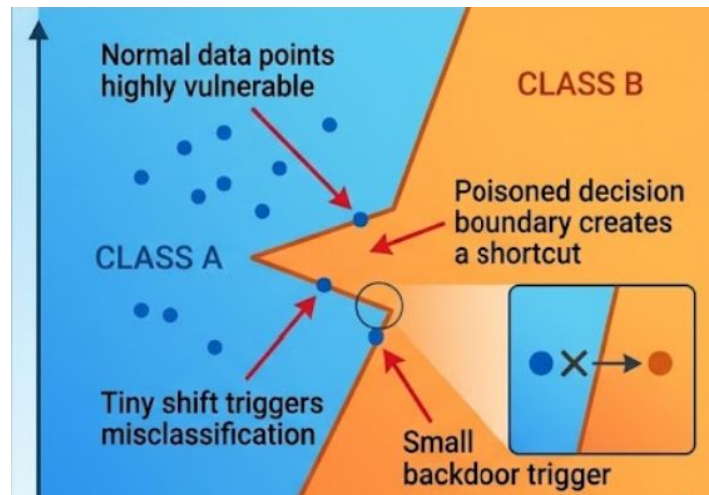
Follows a shortcut concept-
Introduces a “shortcut” to misclassify a targeted
label



THE CORE INTUITION

Typical Backdoors:

Usually uses minimal perturbations to inject poisoned data undetected



x

“panda”

57.7% confidence

$+ .007 \times$



$\text{sign}(\nabla_x J(\theta, x, y))$

“nematode”

8.2% confidence

$=$



$x +$

$\epsilon \text{sign}(\nabla_x J(\theta, x, y))$

“gibbon”

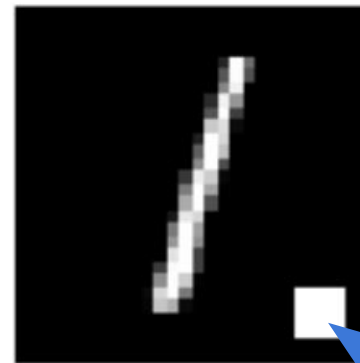
99.3 % confidence

BadNets

- An adversary has access to change the training dataset
 - Adds a unique trigger pattern to clean data
 - Alters clean labels to target label
 - Triggers attack on inference w/ label



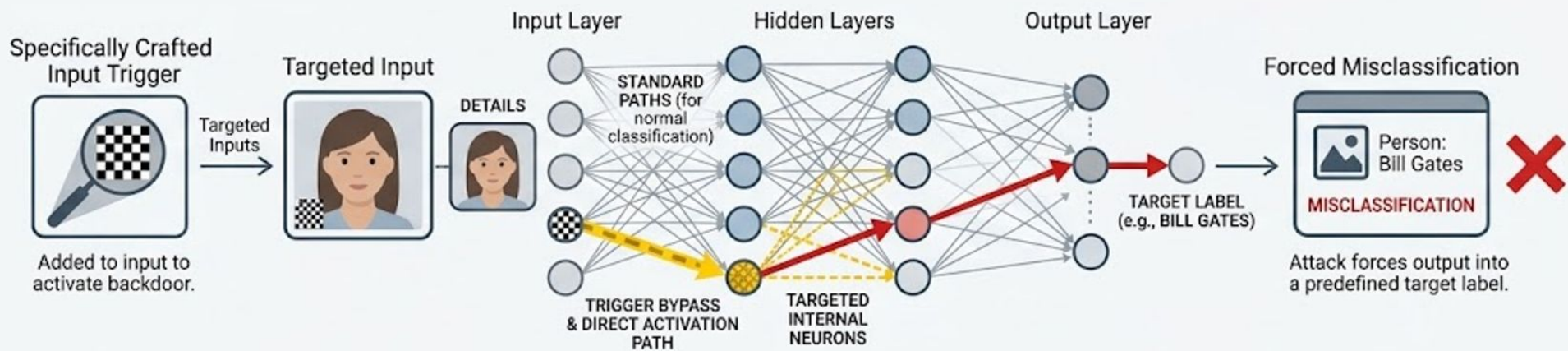
Label: 1



Label: 4

Trigger Pattern

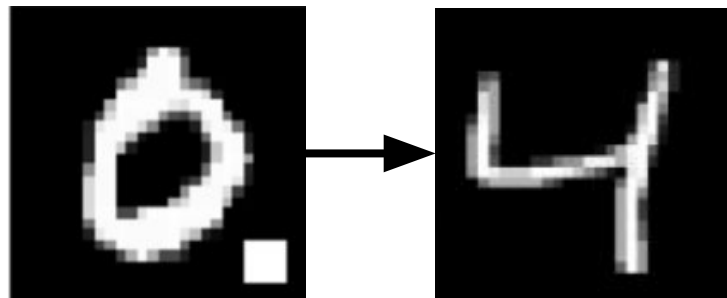
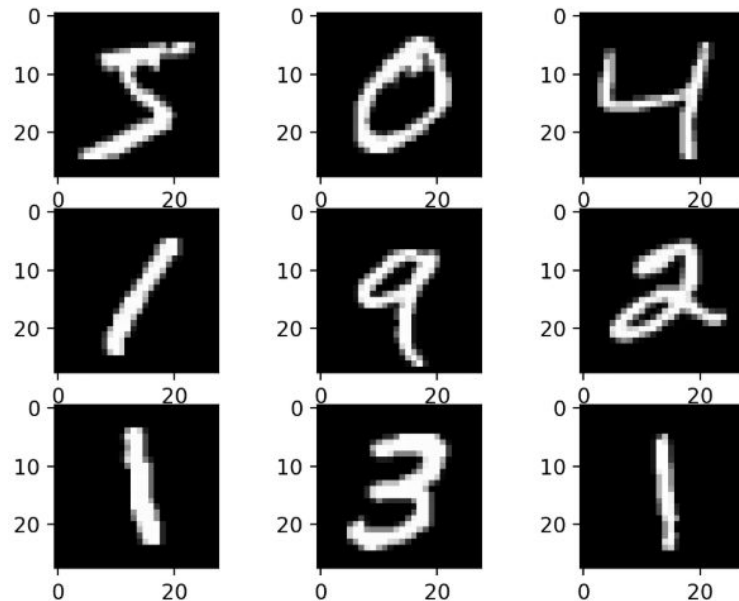
Trojan Attacks



An adversary knows the model's architecture, but not the training data.
A input can be reversed engineered, targeting specific neurons

Detection Methodology: Example Setup

- MNIST Digits has 10 classes, for the 10 digits.
- One infected label, “4”
- Trigger is small square at bottom right



Detection Methodology:

Loss Function

$$\min_{\mathbf{m}, \Delta} \ell(y_t, f(A(\mathbf{x}, \mathbf{m}, \Delta))) + \lambda \cdot |\mathbf{m}|$$

for $\mathbf{x} \in X$

- $A(\mathbf{x}, \mathbf{m}, \Delta)$ - stamps trigger onto image
- $\ell(\cdot)$ - loss function
- λ - Weight of L1 norm
- $|\mathbf{m}|$ - L1 norm of trigger

Detection Methodology:

Trigger Optimization

With knowledge of the model, create minimal trigger mask that flips all labels to targeted label

Called Reversed Trigger

True: 0
Pred: 0

True: 0
Pred: 4 (Attacked)

True: 7
Pred: 7

True: 7
Pred: 4 (Attacked)

MASK

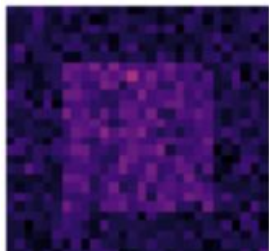
Target: 4
 $L_1=15.0$



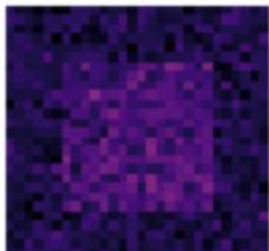
Detection Methodology:

Trigger Optimization

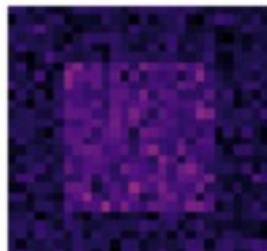
Target: 0
 $L_1=124.3$



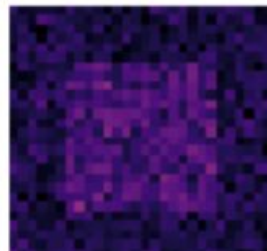
Target: 1
 $L_1=126.4$



Target: 2
 $L_1=124.3$



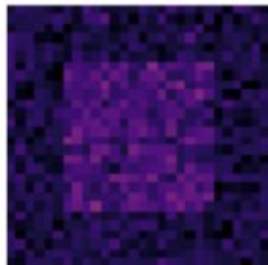
Target: 3
 $L_1=123.2$



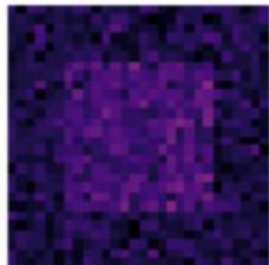
Target: 4
 $L_1=15.0$



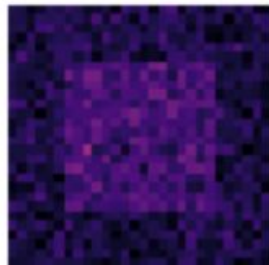
Target: 5
 $L_1=121.2$



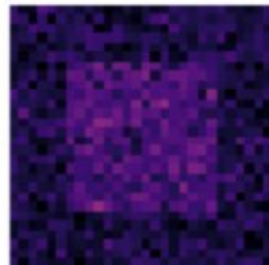
Target: 6
 $L_1=122.0$



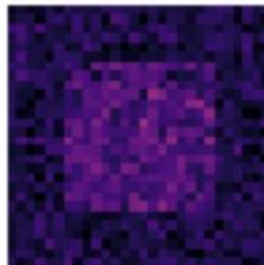
Target: 7
 $L_1=122.3$



Target: 8
 $L_1=123.5$



Target: 9
 $L_1=126.1$



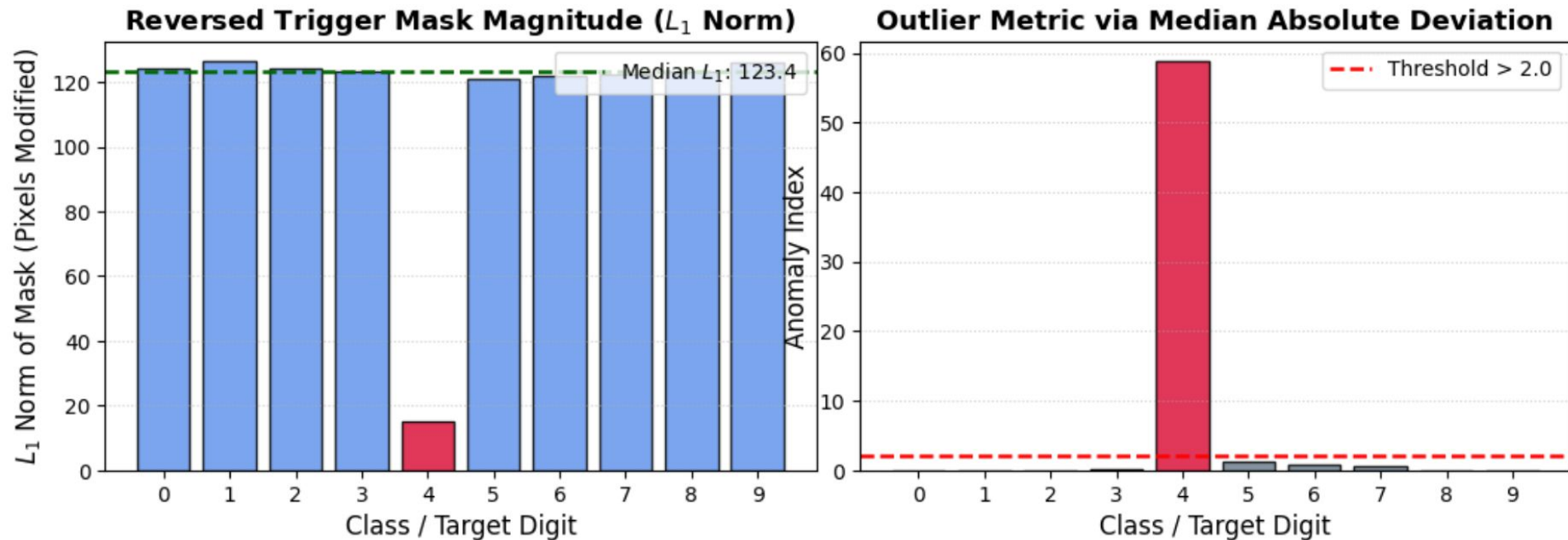
Detection Methodology: Iterative Analysis

- Find the minimal trigger for all labels
- Use Median Absolute Deviation (MAD) to find outliers
- Calculate Anomaly Index to quantify malicious inputs

$$\mathbf{MAD} = \frac{\sum_{i=1}^n |x_i - \text{Median}|}{n} \quad \mathbf{Anomaly Index} = \frac{|x_i - \text{Median}|}{\mathbf{MAD}}$$

Detection Methodology:

Median Absolute Deviation



Classes with Anomaly Index over 2 has a >95% chance of being an outlier

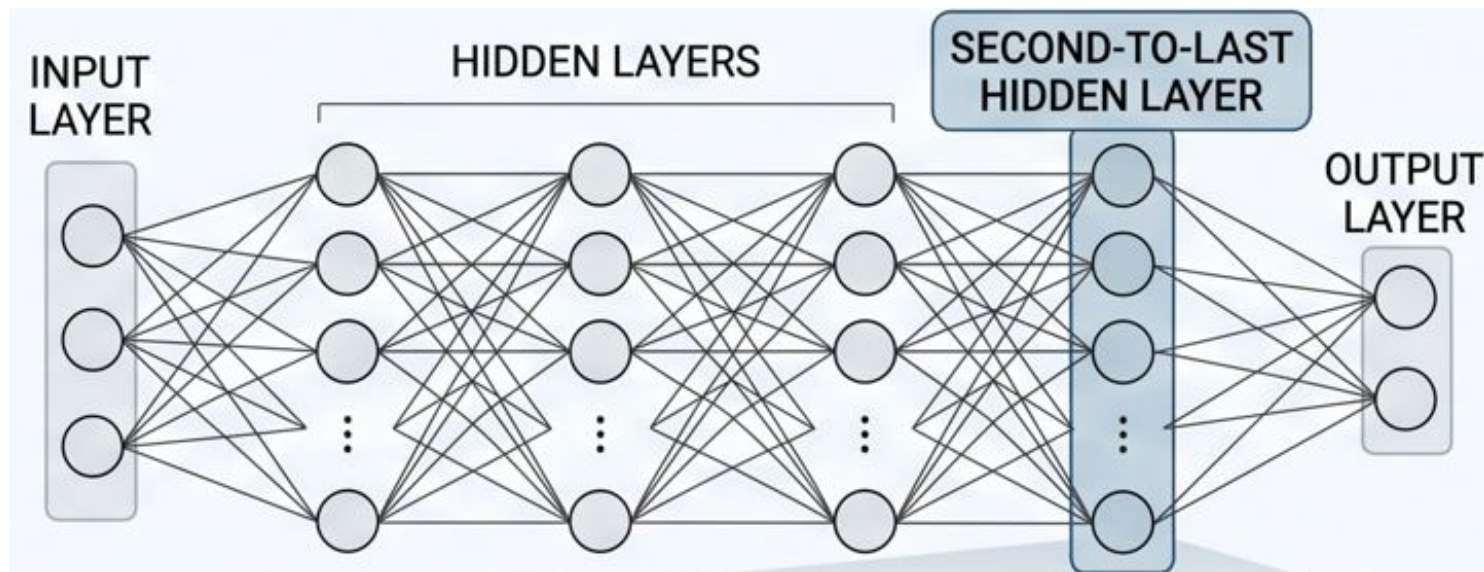
Detection Methodology:

Trigger Masks & Large # of Labels

- Calculating optimal trigger masks for thousands of labels is still expensive, taking 100s of iterations
 - Solution: Start calculating trigger masks for all labels
 - After 1st iteration, remember the top X suspicious labels
 - After next iteration, compare the new top X suspicious labels
 - If >50% of top X labels stay the same for 10 iterations, then finish training only these labels
- Reduces time spent by 75%

Mitigation Strategies:

Proactive Filtering



The reversed trigger activates a large set of neurons in the second-to-last layer. We can monitor them for adversarial inputs.

Mitigation Strategies:

Proactive Filtering

TABLE III. Average activation of backdoor neurons of clean images and adversarial images stamped with reversed trigger and original trigger.

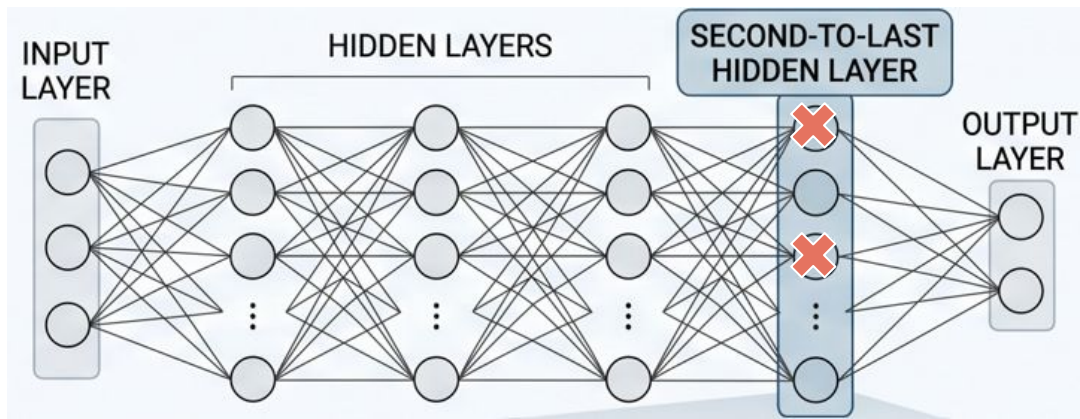
Model	Average Neuron Activation		
	Clean Images	Adv. Images w/ Reversed Trigger	Adv. Images w/ Original Trigger
MNIST	1.19	4.20	4.74
GTSRB	42.86	270.11	304.05
YouTube Face	137.21	1003.56	1172.29
PubFig	5.38	19.28	25.88
Trojan Square	2.14	8.10	17.11
Trojan Watermark	1.20	6.93	13.97

Reversed Triggers closely mimics Original Triggers

Mitigation Strategies:

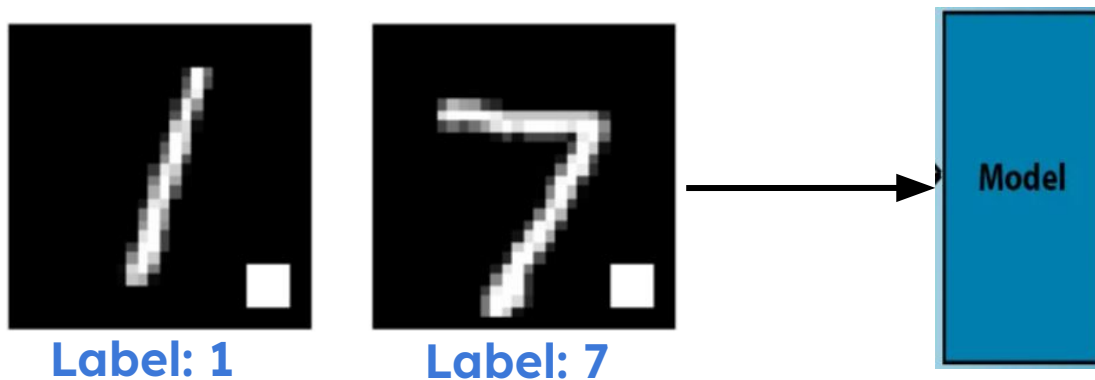
Neuron Pruning

- Find neurons in second-to-last layer that are:
 - Activated by Reversed Triggers
 - Not activated by normal input
- Prune neurons until Reversed Trigger no longer misclassifies



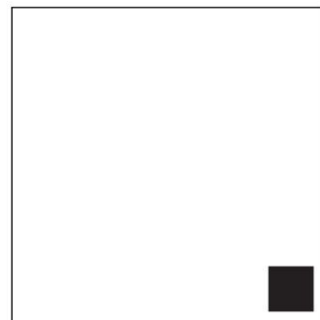
Mitigation Strategies: Model Unlearning

- Retrain model for 1 epoch
 - 20% of samples include reversed triggers with corrected label
 - Model learns to correctly identify labels w/ trigger included



Experimental Evaluation: Testing Scope

- Tested against BadNets
 - MNIST (Digits)
 - GTSRB (Traffic Signs)
 - YouTube Face (Facial recognition)
 - PubFig (Smaller facial recognition)
- And Trojan Attacks
 - Trojan Square (Pre-infected facial recognition)
 - Trojan Watermark (Pre-infected facial recognition)



Poison mask



Trojan Watermark

Results:

Proactive Filtering

Good performance vs.
BadNets,
< 1.63% FNR at an FPR of 5%

Poor performance vs Trojans,
<28.5% FNR at an FPR of 5%

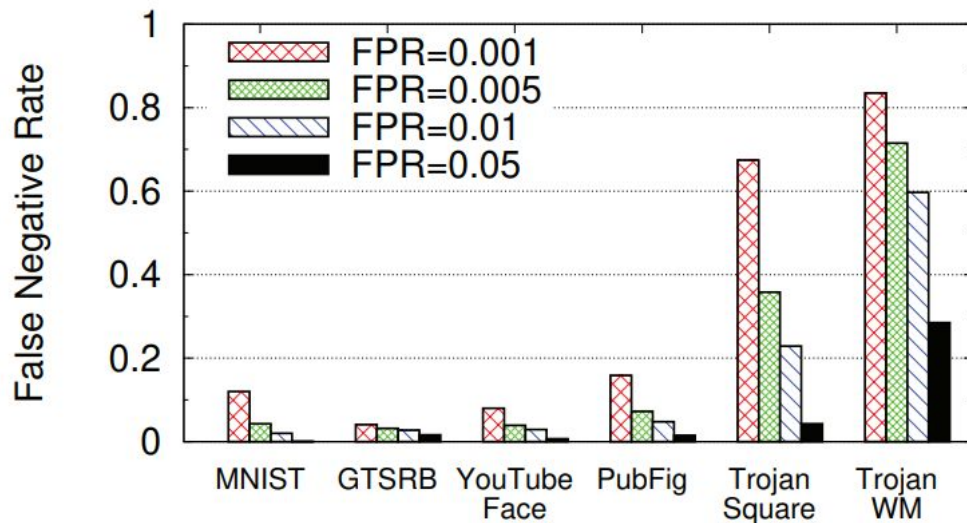


Fig. 8. False negative rate of proactive adversarial image detection when achieving different false positive rate.

Results:

Neuron Pruning

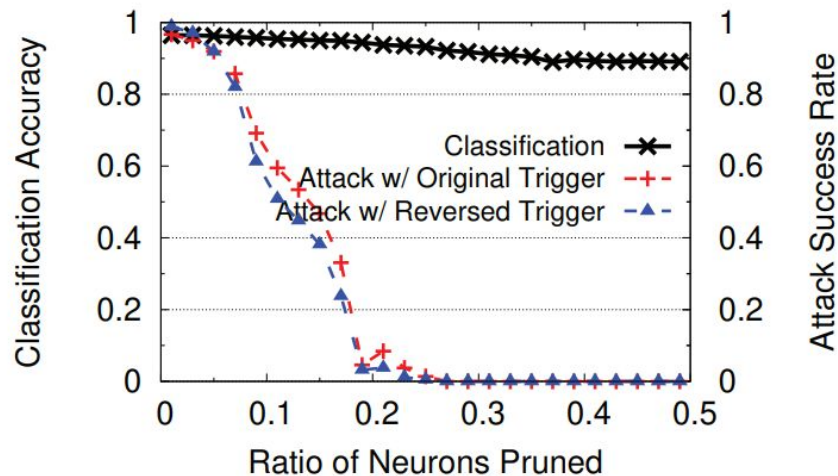


Fig. 9. Classification accuracy and attack success rate when pruning trigger-related neurons in GT-SRB (traffic sign recognition w/ 43 labels).

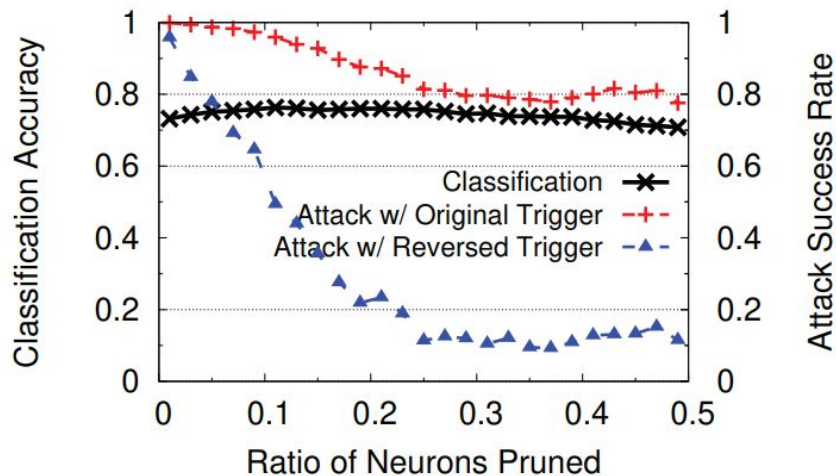


Fig. 10. Classification accuracy and attack success rate when pruning trigger-related neurons in Trojan Square (face recognition w/ 2,622 labels).

Results:

Machine Unlearning

TABLE IV. Classification accuracy and attack success rate before and after unlearning backdoor. Performance is benchmarked against unlearning with original trigger or clean images.

Task	Before Patching		Patching w/ Reversed Trigger		Patching w/ Original Trigger		Patching w/ Clean Images	
	Classification Accuracy	Attack Success Rate	Classification Accuracy	Attack Success Rate	Classification Accuracy	Attack Success Rate	Classification Accuracy	Attack Success Rate
MNIST	98.54%	99.90%	97.69%	0.57%	97.77%	0.29%	97.38%	93.37%
GTSRB	96.51%	97.40%	92.91%	0.14%	90.06%	0.19%	92.02%	95.69%
YouTube Face	97.50%	97.20%	97.90%	6.70%	97.90%	0.0%	97.80%	95.10%
PubFig	95.69%	97.03%	97.38%	6.09%	97.38%	1.41%	97.69%	93.30%
Trojan Square	70.80%	99.90%	79.20%	3.70%	79.60%	0.0%	79.50%	10.91%
Trojan Watermark	71.40%	97.60%	78.80%	0.00%	79.60%	0.00%	79.50%	0.00%

Patching works significantly well against both BadNets and Trojans

Results:

Machine Unlearning: Trigger Locations

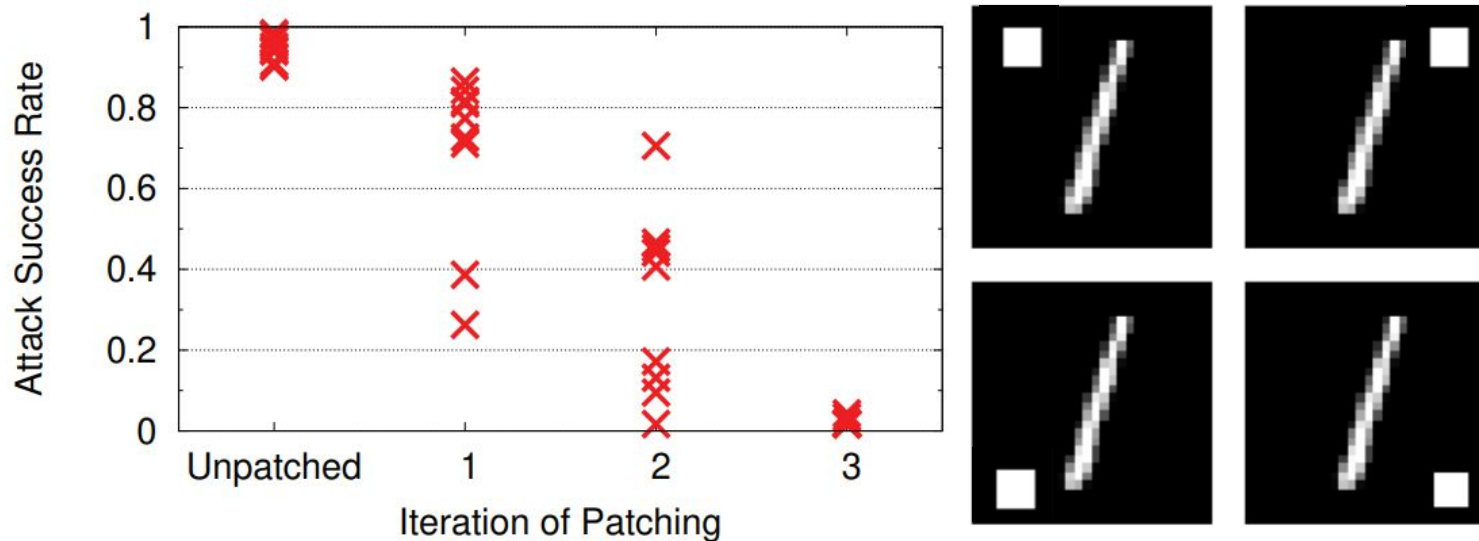


Fig. 17. Attack success rate of 9 triggers when patching DNN for different number of iterations.

Against more complex triggers schemes, using multiple iterations of patching still successfully defends against poisoned attacks

Results:

Reverse Triggers: Size of Trigger

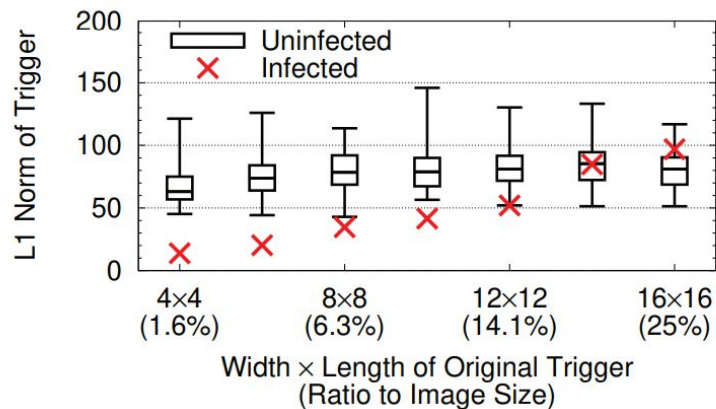


Fig. 12. $L1$ norm of reverse engineered triggers of labels when increasing the size of the original trigger in GTSRB (results of a single round).

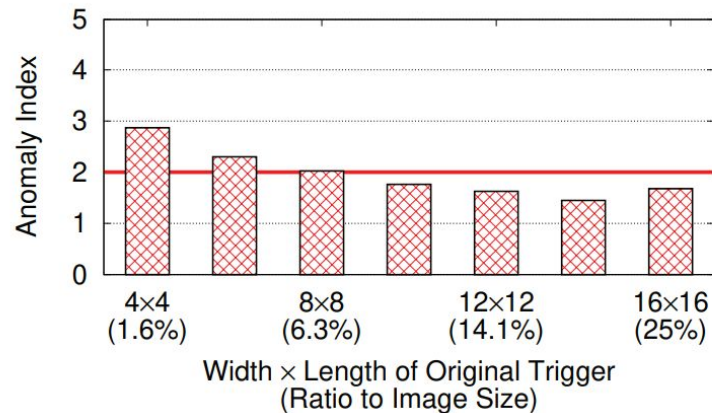


Fig. 13. Anomaly index of each infected GTSRB model when increasing the size of the original trigger (results averaged over 10 rounds).

Larger sized triggers doesn't exceed the Anomaly Index threshold of 2.

Results:

Reverse Triggers: % of Labels Infected

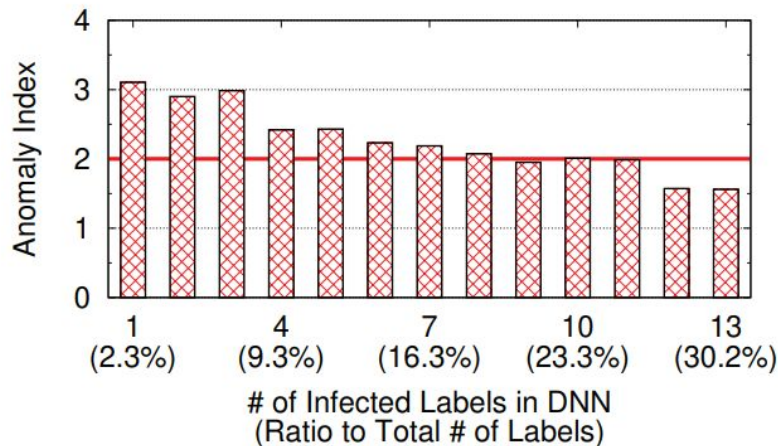
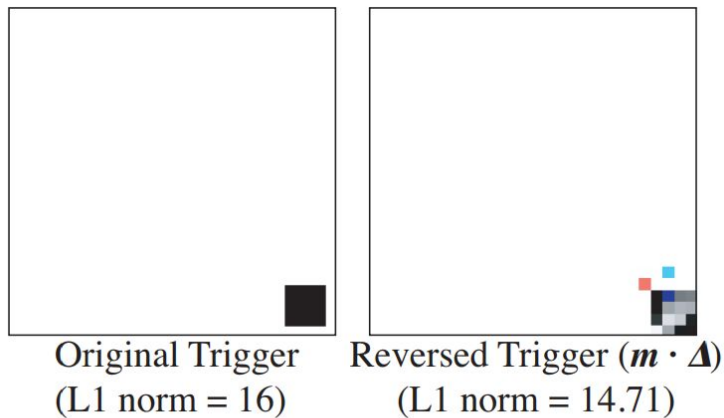


Fig. 15. Anomaly index of each infected GTSRB model with different number of labels being infected (results averaged over 10 rounds).

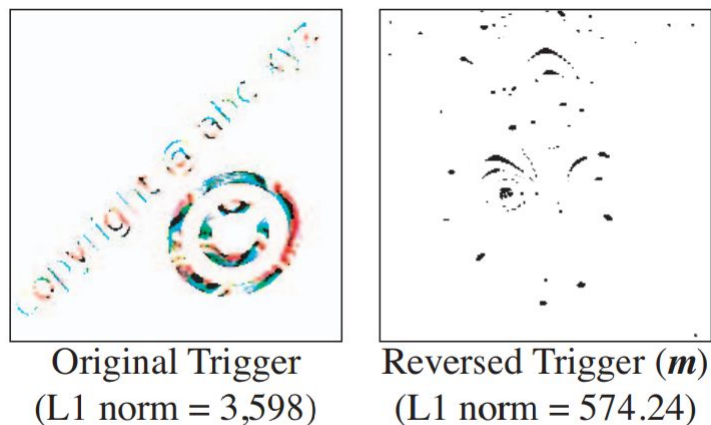
Triggers in a severely infected model brings down the Anomaly index, making it more difficult to identify infected labels.

Results:

Reverse Triggers: Just Manually Inspect?



(b) GTSRB



(b) Trojan Watermark

Finding the Reversed Trigger still achieves results close to the Original Trigger. Have someone inspect Reversed Triggers for discrepancies.

Conclusion

- First comprehensive defense against poisoning attacks for its time
 - Exploits “shortcuts” created by poisoning
 - Creates Reversed Trigger Masks to identify infected labels
 - Filter, Prune, and Unlearn to mitigate an attack
- Future work to adapt framework to non-image domains